

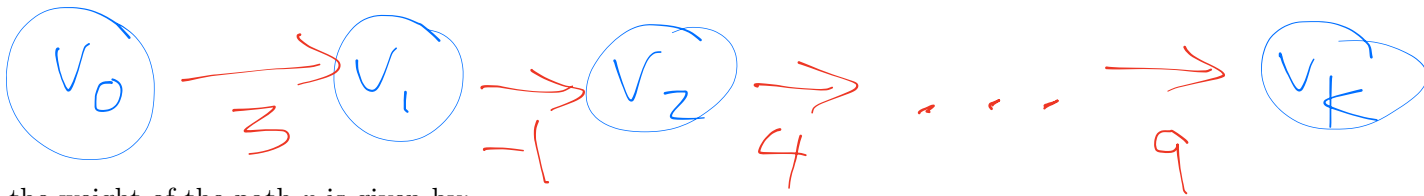
Course Logistics

- Reading from Chapter 24 this week, especially sections 24.1, and 24.3
- Homework 3 due Friday

1 The Single-Source (weighted) Shortest Paths Problem

Consider a directed and weighted graph $G = (V, E, w)$, where $w: E \rightarrow \mathbb{R}$ maps edges to weights.

Given a path $p = \{v_0, v_1, \dots, v_k\}$ from $i = v_0$ to $j = v_k$,



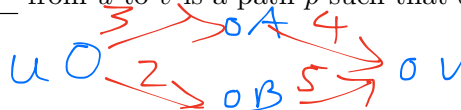
the weight of the path p is given by:

$$w(p) = \sum_{t=1}^k w(v_{t-1}, v_t) = 3 + (-1) + 4 + \dots + 9$$

Let $\mathcal{P}_{ij}$ be the set of paths from nodes i to j . The *shortest path weight* from i to j is:

$$\delta(i, j) = \begin{cases} \min w(p) & \text{if there is a path } i \text{ to } j \\ \infty & \text{otherwise} \end{cases}$$

A shortest path from u to v is a path p such that $\delta(u, v) = w(p)$.



Definition Given a *source* node $s \in V$, the single-source shortest paths problem (SSSP) seeks to find the shortest path weight $\delta(s, u)$ for every $u \in V$.

1.1 Shortest Path Properties

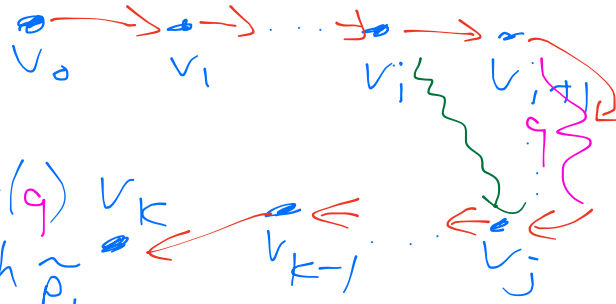
Triangle Inequality For any set of three nodes s , u , and v ,

$$\delta(s, u) \leq \delta(s, v) + \delta(v, u)$$

Optimal Substructure

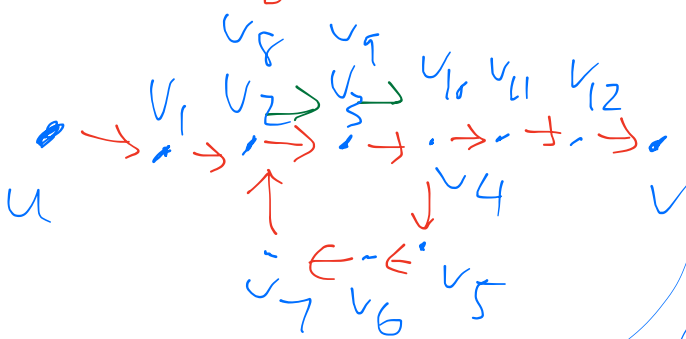
Lemma 1.1. If $p = \{v_0, v_1, \dots, v_k\}$ is a shortest path from v_0 to v_k , then all of its subpaths are shortest paths

Proof: If not, there is v_i, v_j for which there exists $\tilde{p}$ from v_i to v_j so that $w(\tilde{p}) < w(q)$. But then if we replace q with $\tilde{p}$, we have a smaller total weight, so p is not



Cycles and shortest paths optimal, which is a contradiction

Lemma 1.2. : If $G = (V, E, w)$ has no negative edges, then for every pair of nodes, there is a shortest path with no cycles.



Proof: Let p be a shortest path from u to v . If p contains a cycle C , then $w(C) \geq 0$.

If $w(C) > 0$, then p cannot be optimal because C would decrease cost. Otherwise, then removing C produces shortest path with no cycles

1.2 Negative edges

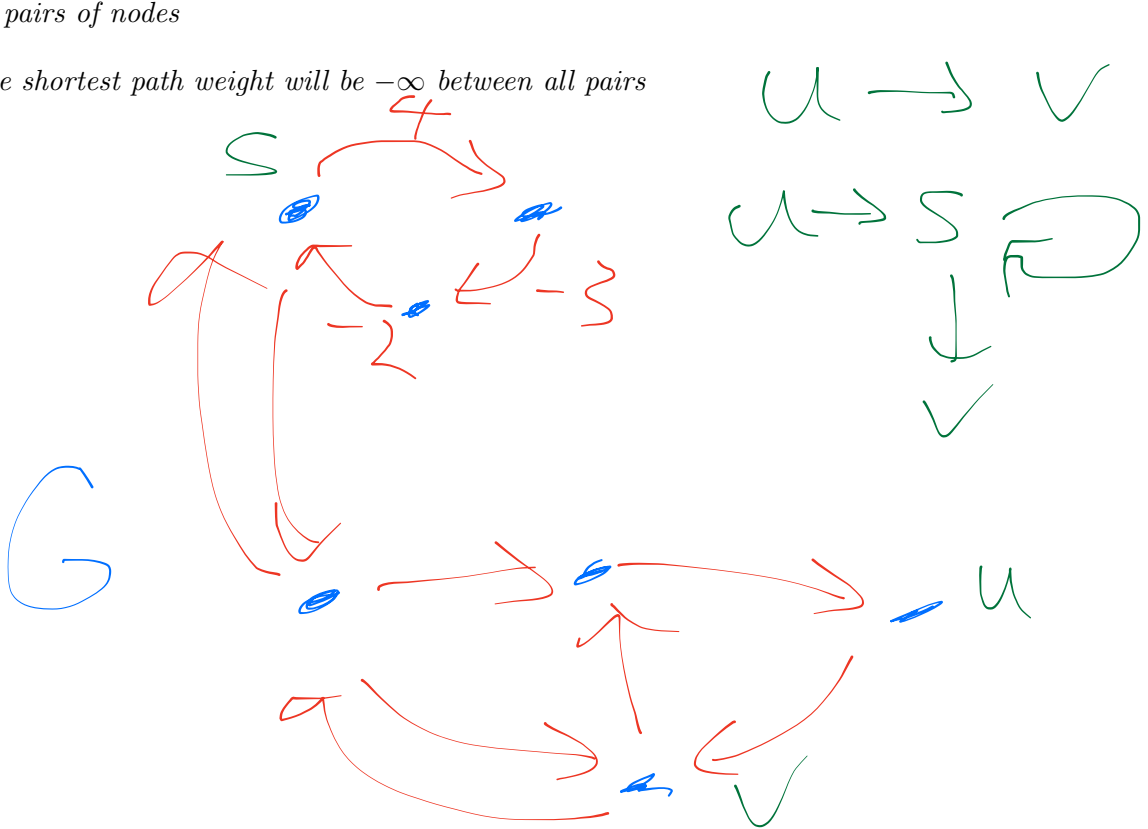
Question 1. Assume $G = (V, E, w)$ contains negative edges, and that it is strongly connected. If G contains a cycle where the sum of edge weights is negative, which of the following is true?

A The shortest path weight between every pair of nodes will be a finite negative number

B The shortest path weight between some (but not necessarily all nodes) will be a finite negative number

C The shortest path weight can be $-\infty$ for some pairs of nodes, but not necessarily all pairs of nodes

D The shortest path weight will be $-\infty$ between all pairs



1.3 SSSP Algorithm Basics

Let s denote the starting node of an SSSP problem. In shortest path algorithms, we maintain two attributes for each $v \in V$:

Attribute	Explanation	Initialization	Invariant
$u.d$	best guess for distance from s to u	$s.d = 0$ $u.d = \infty$ for $u \neq s$	$u.d \geq \delta(s, u)$
$u.\pi$	parent of u in path from s to u	$u.\pi = \text{NULL}$	$u.\pi$ is updated whenever $u.d$ is updated



Given values for these attributes, we can construct a *predecessor* graph:

- $V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}$ All nodes with parent, plus s

- $E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}$ All edges connecting nodes and parents

If $v.d = \delta(s, v)$ and $v.\pi$ gives the predecessor of v in a shortest path from s to v , then $G_\pi = (V_\pi, E_\pi)$ is called a Shortest path tree.

1.4 Relaxation and Generic Algorithm

The key idea behind shortest path algorithms is to successively “relax” edges.

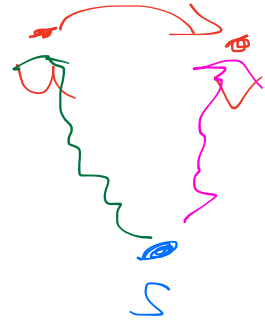
RELAX(u, v, w)

if $v.d > u.d + w(u, v)$ then

$$v.d = u.d + w(u, v)$$

$$v.\pi = u$$

end if



GENERICSSSP(G, w, s)

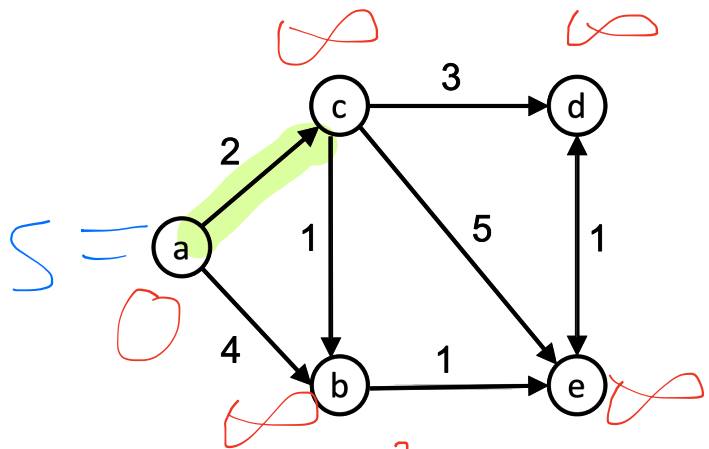
INITIALIZE-SSSP(G, w, s)

while There is some edge (u, v) with $v.d > u.d + w(u, v)$ do

Relax(u, v, w)

end while

} while some edge can be relaxed, relax that edge



Relax($a, c, 2$)

$$a.d + w(a, c)$$

$$< c.d$$

$$c.d = 2,$$

$$c.\pi = a$$

Relax($c, d, 3$)

$$c.d + w(c, d) < d.d$$

$$\overset{11}{2} + \overset{11}{3} = 5$$

$$d.d = 5, d.\pi = c$$

Relax($c, b, 1$)

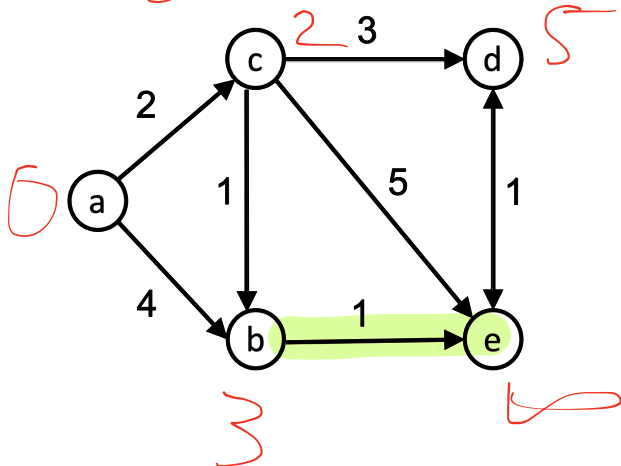
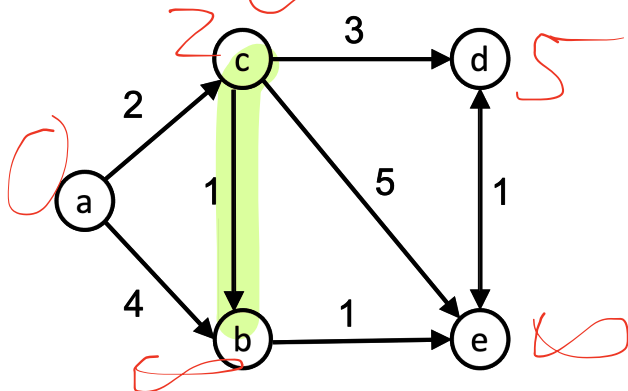
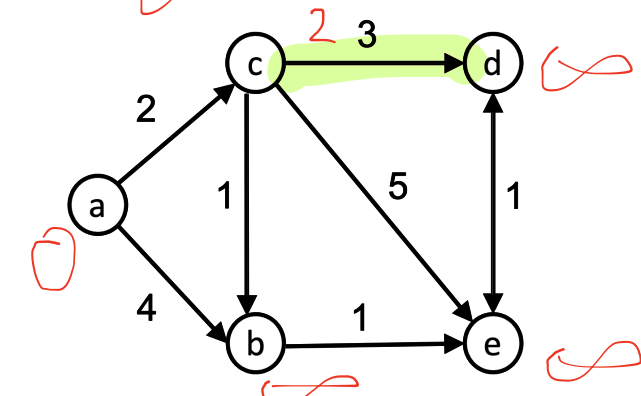
$$b.d = 3, b.\pi = c$$

Relax($b, e, 1$)

$$e.d = b.d + w(b, e)$$

$$= 3 + 1 = 4$$

$$e.\pi = b$$



2 The Bellman Ford Algorithm

Let's do a quick recap of what we have learned so far about the SSSP problem.

Cycles Let p be a cycle in a directed graph.

- Case 1: $w(p) \geq 0$: then I can get rid of the cycle and I have a better path
- Case 2: $w(p) < 0$: then all paths are of length $-\infty$ because we can traverse p as many times as we want to decrease the path weight.

If Case 2 is true, we would prefer to be aware of this, in which case we state that all shortest paths have weight $-\infty$ and we're done.

Otherwise, we do not seek out paths with cycles when solving the SSSP problem.

RELAX(u, v, w)

```
if  $v.d > u.d + w(u, v)$  then
     $v.d = u.d + w(u, v)$ 
     $v.\pi = u$ 
end if
```

GENERICSSSP(G, w, s)

```
INITIALIZE-SSSP( $G, w, s$ )
while There is some edge  $(u, v)$  with  $v.d > u.d + w(u, v)$  do
    RELAX( $u, v, w$ )
end while
```

Relaxations and Generic Algorithm Now we are ready to find a more concrete implementation of this approach, by answering the question:

how many times do we
need to relax?

2.1 Bellman-Ford Algorithm Pseudocode

BELLMAN-FORD(G, w, s)

for $v \in V$ do

$v.d = \infty$

$v.\pi = \text{NIL}$

end for

$s.d = 0$

} initialization

for $i = 1$ to $|V| - 1$ do

 for $(u, v) \in E$ do

 RELAX(u, v, w)

 end for

end for

← loop over all edges
← relax

} repeated $V-1$ times

for $(u, v) \in E$ do

 if $v.d > u.d + w(u, v)$ then

 Return "the graph has a negative cycle"

 end if

end for

} one more loop,
to check for
negative weight
cycles

$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}$

$E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}$

Return (V_π, E_π) and $\{v.d : v \in V\}$

Question 2. What is the runtime complexity of running the BELLMAN-FORD Algorithms?

A $O(V + E)$

B $O(E \log V)$

C $O(E + V \log V)$

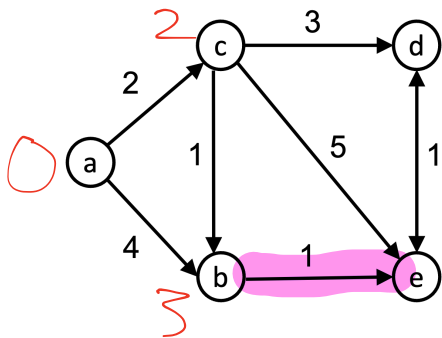
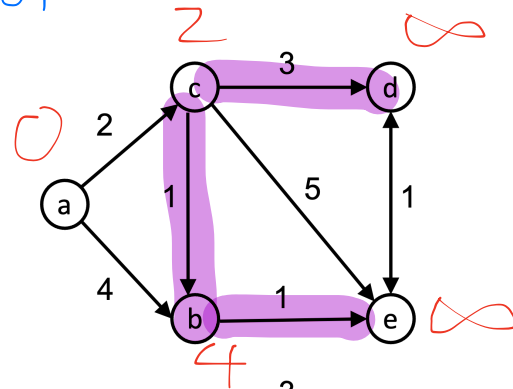
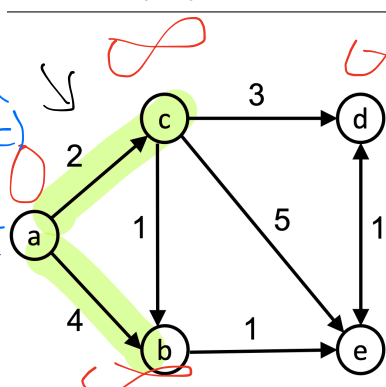
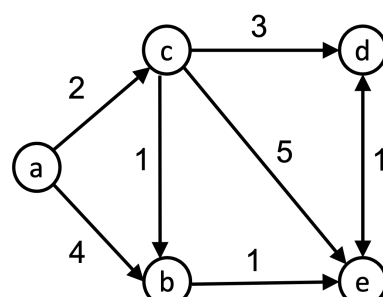
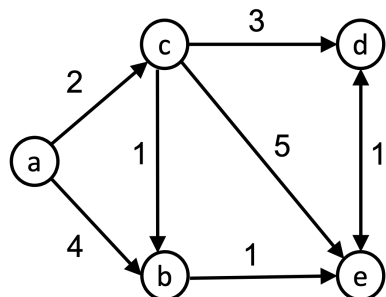
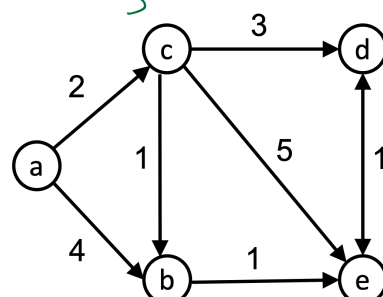
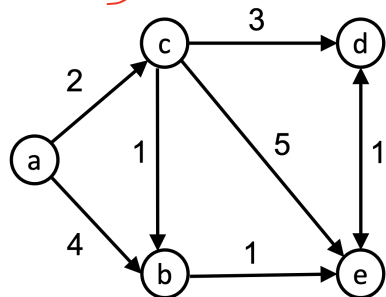
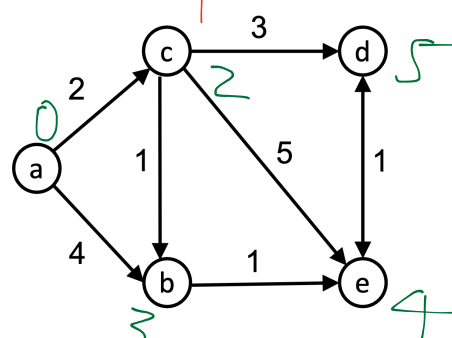
D $O(EV)$

$$\text{BELLMAN-FORD}(G, w, s)$$
INITIALIZESSSP(G, w, s)**for** $i = 1$ **to** $|V| - 1$ **do****for** $(u, v) \in E$ **do**
$$\text{RELAX}(u, v, w)$$

end for

end for

Return $\{v.d\}$


$$\begin{aligned} w(e, d, 1) \\ d \cdot d = 5 \\ w(e, d) = 1 \\ e \cdot d = 4 \end{aligned}$$


does not
hold

$\text{relax}(x, e, l)$ does not hold
 $d.d = \infty$ $e.d = \infty$
 $w(d, e) = 1$ $e.d > d.d + w(d, e)$

$$S = \frac{1}{2} \ln \frac{a \cdot d}{b \cdot c}$$
$$\begin{aligned} b.d &= 3 \\ w(b,e) &= 1 \\ e.d &= 5 \end{aligned}$$
$$S = \sqrt{0}$$

U

Page 9

$$V_2$$

$$V \supseteq$$


2.2 Correctness

Theorem 2.1. Assuming that $G = (V, E, w)$ has no negative edges, when BELLMAN-FORD terminates, $v.d$ will equal $\delta(s, v)$ for every $v \in V$.

Proof: Consider a shortest path from s to any vertex v . $p = \{v_0, v_1, \dots, v_k\}$ ^{$\uparrow$ p}
 After 1st round of relaxations, $v_1.d = \delta(s, v_1)$
 $\underbrace{v_1.d}_{\text{running estimate}} = \underbrace{\delta(s, v_1)}_{\text{true distance}}$

After 2nd round of relaxations, $v_2.d = \delta(s, v_2)$.

After k rounds, $v_k.d = \delta(s, v_k)$.

Since p does not cycles (no negative weight cycles), then $k \leq |V| - 1$, so correct answer after $|V| - 1$ rounds.

Runtime: $O(V E)$

2.3 Catching Negative Cycles

BELLMAN-FORD (Negative cycle check)

Do everything else...

for $(u, v) \in E$ do

if $v.d > u.d + w(u, v)$ then

Return an error, the graph has a negative cycle

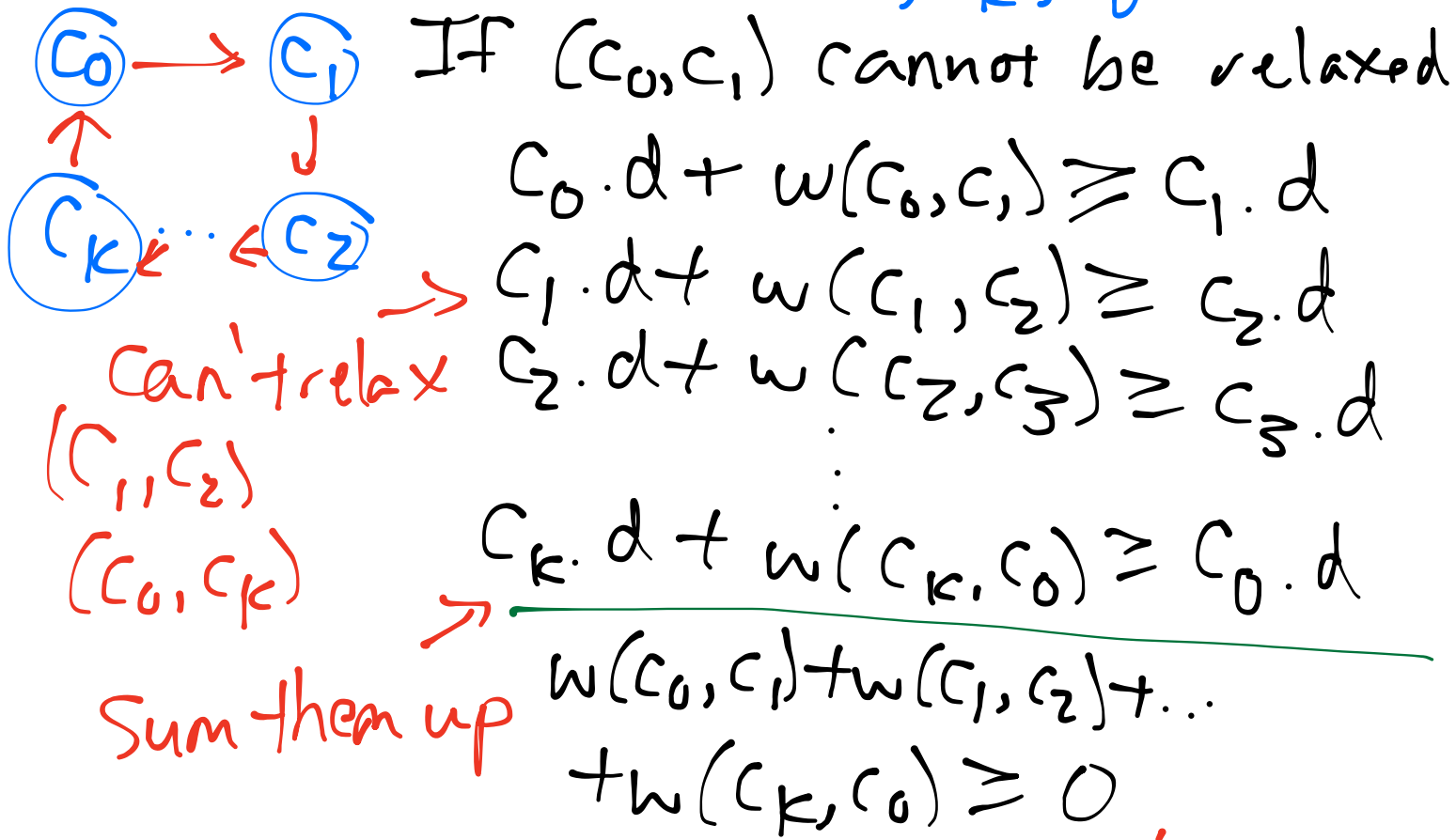
end if

end for

Theorem 2.2. If $G = (V, E, w)$ has a negative weight cycle that is reachable from s , then the code above will produce an error

Proof: Negative weight cycle c

has edges $c_0, c_1, \dots, c_k, c_0$



Contradicting c is a negative weight cycle

3 Dijkstra's Algorithm

DIJKSTRA(G, w, s)

INITIALIZE-SSSP(G, w, s)

$S = \emptyset$

$Q = V$

while $Q \neq \emptyset$ **do**

$u = \text{EXTRACTMIN}(Q)$

$S = S \cup \{u\}$

for $v \in \text{Adj}[u]$ **do**

$\text{RELAX}(u, v, w)$

end for

end while

$$\delta(s, s) = 0$$

m-heap

← extract min undetermined vertex

← update its neighbor

This algorithm maintains:

$S =$ "running estimates are correct"

- S = set of vertices for which $v.d = \delta(s, v)$

- Q = min-priority queue storing $v.d$ for all vertices in $V - S$

← remaining vertices

Question 3. How many times does the while loop in Dijkstra's algorithm iterate?

A $O(V)$ times

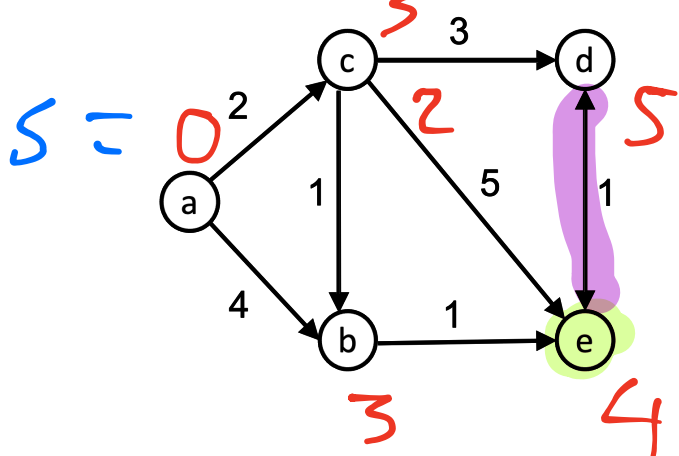
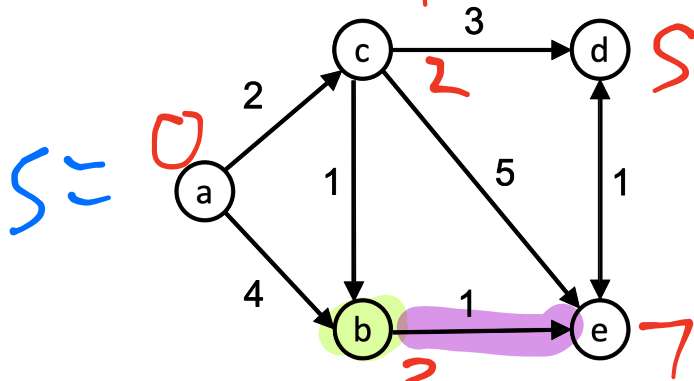
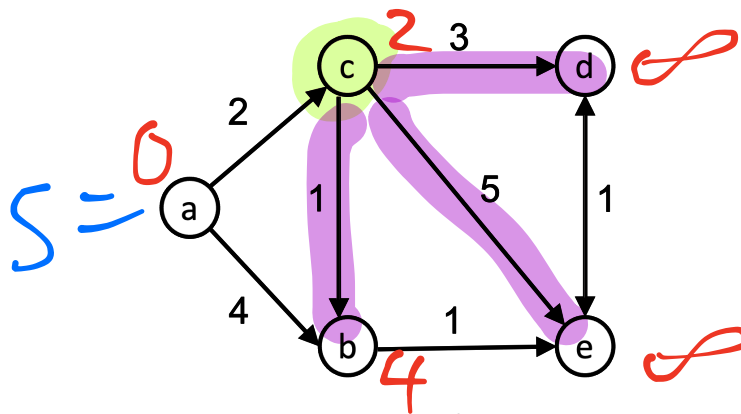
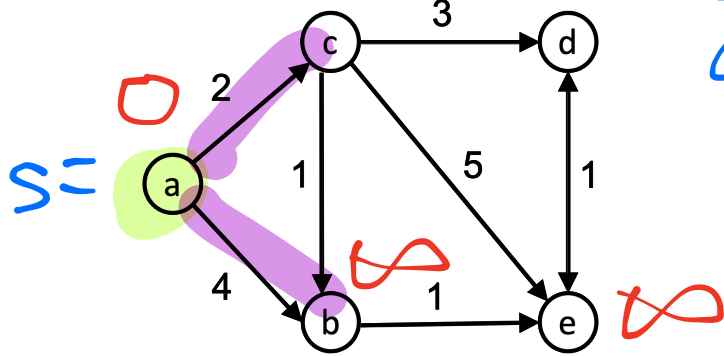
B $O(E)$ times

C $O(EV)$ times

D Depends on the graph



3.1 Example



$a.\pi = b.\pi = c.\pi = d.\pi$
 $= e.\pi = \text{Null}$
 $\{0, \infty, \infty, \infty, \infty\}$

$u = a$

$c.d = 2, b.d = 4$

$c.\pi = a, b.\pi = a$

$\{2, 4, \infty, \infty\}$

$b.d = 3, d.d = 5, e.d = 7$

$b.\pi = c, d.\pi = c,$
 $e.\pi = c$

$\{3, 5, 7\}$

$e.d = 4, e.\pi = b$

$\{4, 5\}$

done

$\{5\}$

3.2 Correctness

```
DIJKSTRA( $G, w, s$ )
  INITIALIZE-SSSP( $G, w, s$ )
   $S = \emptyset$ 
   $Q = V$ 
  while  $Q \neq \emptyset$  do
     $u = \text{EXTRACTMIN}(Q)$ 
     $S = S \cup \{u\}$ 
    for  $v \in \text{Adj}[u]$  do
      RELAX( $u, v, w$ )
    end for
  end while
```

We will take the following properties as given.

Upper bound property For each $v \in V$, the property $v.d \geq \delta(s, v)$ is maintained throughout the algorithm.

$v.d = \infty$ or $v.d$ is based on path from s to v

Convergence property Let $p = \{s, \dots, u, v\}$ be a shortest path involving an edge $(u, v) \in E$. If $u.d = \delta(s, u)$ at any time prior to relaxing edge (u, v) , then $v.d = \delta(s, v)$ after relaxing (u, v) .



No path property If there is no path from s to v , then $v.d = \delta(s, v) = \infty$ throughout the algorithm.

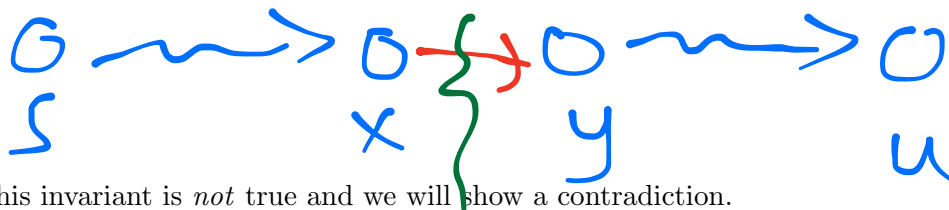
Theorem 3.1. Dijkstra's algorithm, run on a weighted, directed graph $G = (V, E, w)$ with positive weights and source $s \in V$, terminates with $u.d = \delta(s, u)$ for all $u \in V$

Proof. We maintain the following invariant throughout.

Invariant: At the start of the while loop, the node u added to S satisfies $u.d = \delta(s, u)$.

If this invariant is true, the theorem is shown. Why?

node we extract estimate is correct



Assume this invariant is *not* true and we will show a contradiction.

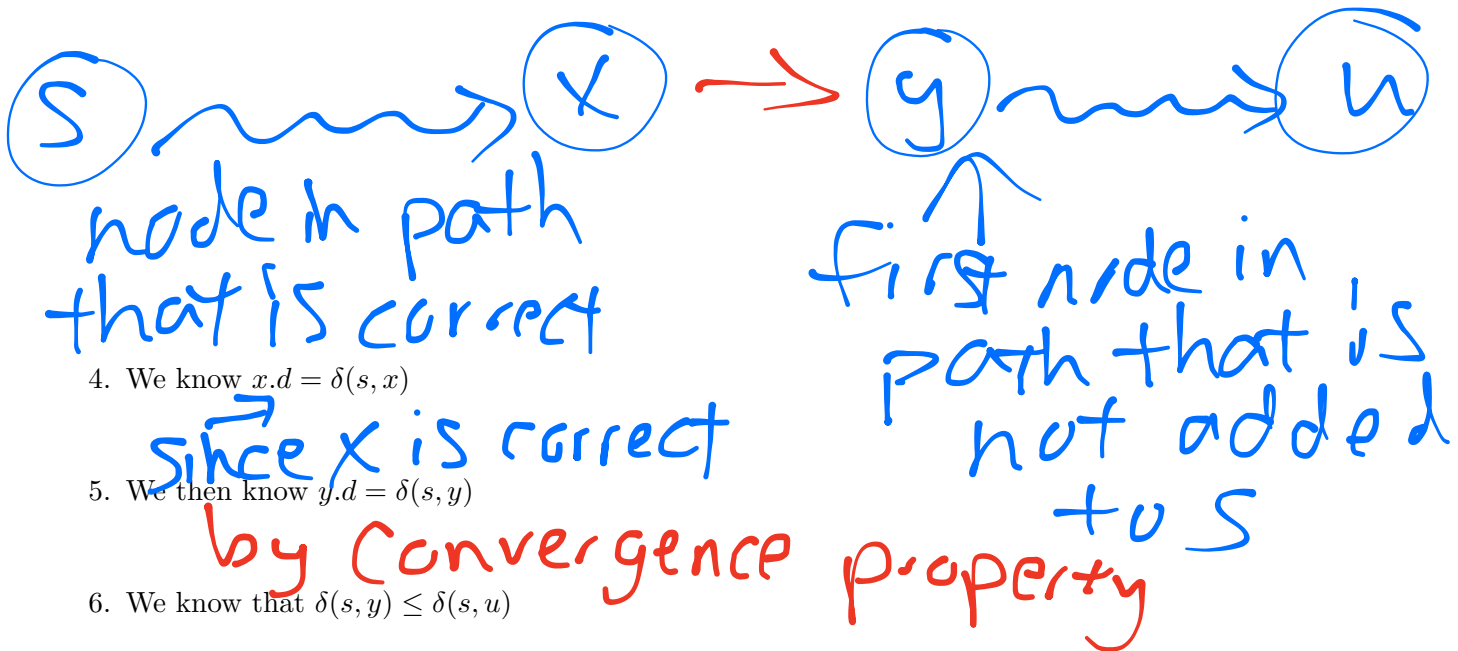
Define u to be the first node added to S where $u.d \neq \delta(s, u)$.

1. We know $u \neq s$. since $s.d = 0 = \delta(s, s)$

2. We know there is a path from s to u .

since $s \neq u$

3. Let p be a shortest path from s to u , which goes from S to $V - S$, let y be the first node in p from $V - S$ in this path, and x be its predecessor.



4. We know $x.d = \delta(s, x)$

since x is correct

5. We then know $y.d = \delta(s, y)$

by convergence property

6. We know that $\delta(s, y) \leq \delta(s, u)$

7. And we know $\delta(s, u) \leq u.d$

8. We know also that $u.d \leq y.d$

9. The last steps show $y.d = \delta(s, y) \leq \delta(s, u) \leq u.d \leq y.d$. Why is this a contradiction?

this implies y is extracted before u , but then $y.d$ is correct, which contradicts y being in $V \setminus S$

3.3 Runtime Analysis

```
INITIALIZE-SSSP( $G, w, s$ )
 $S = \emptyset; Q = V$ 
while  $Q \neq \emptyset$  do
     $u = \text{EXTRACTMIN}(Q)$ 
     $S = S \cup \{u\}$ 
    for  $v \in \text{Adj}[u]$  do
        if  $v.d > u.d + w(u, v)$  then
             $v.d = u.d + w(u, v)$ 
             $v.\pi = u$ 
        end if
    end for
end while
```

Question 4. Assuming that all nodes are reachable from s , what runtime we can obtain for Dijkstra's algorithm? (Warning: there may be more answer that you can argue is valid! Don't just select an answer, think about a specific reasoning for the runtime analysis you select).

A $O(V + E)$

B $O(V^2)$

C $O(VE)$

D $O(E \log V)$

E $O(E + V \log V)$

(y is incorrect
when popped by
Dijkstra's)

min heap

with Fibonacci
heap

3.4 Class Activity: another example

